Appendix K — Database & transactions fundamentals (Tier 3 follow-ups)

Source: `MICROSOFT_DATABASES_TRANSACTIONS_DEEP_DIVE.md`

Microsoft — Senior Backend Engineer: Transactions, SQL and Data-Layer Deep Dive

Round type: not a coding round — transactions, SQL, and the data layer, in detail. **Companion:** your Hello Interview checklist (Microsoft · Backend Engineer, Senior). Chapter 15 maps every topic below to the exact Hello Interview page, in the order to read them.

How to use this doc. Each chapter is: the mechanism → the failure it prevents → a small concrete example → the question you'll be asked. The ⚠ blocks are the things people get subtly wrong. The "Try it yourself" blocks are for **you to write and run** against a local Postgres — typing the two-session experiments is how this sticks; reading them does not. (Chapter 16 is the drill plan.)

Two engines, one set of ideas. Microsoft interviewers think in **SQL Server / Azure SQL**; most public material and your own ResolveFlow work is **PostgreSQL**. The concepts are identical; the defaults differ. Every chapter flags the difference where it matters (Chapter 9 collects them).

Progress tracker (updated 2026-10-03, after dealing-with-contention; multi-step done; Temporal next)

Self-reported by Bidisha; update the boxes as you go. Interview date/round structure: **still unknown**.

Hello Interview — database / system-design reading

Status	Topic	HI path
✅ Done	Data Modeling	`core-concepts/data-modeling`
✅ Done	Database Indexing	`core-concepts/db-indexing`
✅ Done (shaky: cross-shard breakage + logical-shard resharding needed re-teaching)	Sharding	`core-concepts/sharding`
▶ Next	Consistent Hashing	`core-concepts/consistent-hashing`
⬜ Left	CAP Theorem	`core-concepts/cap-theorem`
⬜ Left	Scaling Reads	`patterns/scaling-reads`
⬜ Left	Scaling Writes	`patterns/scaling-writes`

Status	Topic	HI path
✅ Done	Caching	`core-concepts/caching`
⬜ Left	Numbers to Know	`core-concepts/numbers-to-know`
❓ Unconfirmed	PostgreSQL deep dive (ACID/SQL appendix)	`deep-dives/postgres`
✅ Done	Dealing with Contention	`patterns/dealing-with-contention`
✅ Done	Multi-step Processes	`patterns/multi-step-processes`
▶ Next	Temporal, CDC	`deep-dives/temporal`, `…/change-data-capture`
⬜ Left	DynamoDB, Cassandra, Kafka, Redis, ZooKeeper	`deep-dives/*`
⬜ Left	Problem breakdowns (Ticketmaster, Payment, Auction, Robinhood, Gopuff, Job Scheduler, Flash Sale) + 3 easy / 3 medium / 2 hard guided practices	
⬜ Left	Quizzes (2 of 67 done as of 2026-10-02)	

This doc's own chapters: written, **not yet drilled** — Ch. 1–14 are reference; the two-session experiments (§3), the SKIP LOCKED queue, deadlock reproduction, EXPLAIN drills (§7) and the 8 SQL problems (§9.8) are all still to be *run by you*. Ch. 16's 7-day plan has not started.

Other tracks on the dashboard: Coding readiness 0% (not part of this round, but on the checklist); Behavioral readiness 0% (story catalog / CARL not started).

No doc exists yet for the six "next" topics (sharding, consistent hashing, CAP, scaling reads/writes, caching, numbers to know) — a study list was given in chat only; §11 here covers them briefly. Write a dedicated chapter only if asked.

---

1. Transactions and ACID — what each letter actually promises

A **transaction** is a unit of work that the database treats as all-or-nothing and isolated from other work. ACID is four *different* guarantees implemented by four *different* mechanisms:

	Guarantee	Mechanism	What breaks without it
Atomicity	All statements commit, or none do	Undo information / rollback (Postgres: old row versions stay invisible; SQL Server: undo from the log)	Debit succeeds, credit fails → money vanishes
Consistency	A commit moves the DB from one *valid* state to another	**Constraints** (PK, FK, UNIQUE, CHECK, NOT NULL) + triggers + *your application invariants*	Negative balance, orphan rows
Isolation	Concurrent transactions don't see each other's intermediate states (to a configurable degree)	MVCC and/or locks (§3–4)	Lost updates, double-booking
Durability	Once COMMIT returns, the data survives a crash	**Write-ahead log (WAL)** flushed (`fsync`) before COMMIT returns (§2)	Acknowledged write disappears after power loss

⚠ **"C" is the odd one out.** A, I and D are properties the *database* provides. Consistency is a joint property: the DB enforces the constraints you declare; *you* are responsible for declaring them and for application-level invariants the DB can't see. "The database guarantees consistency" is the answer that gets pushed back on. Say: "*the database enforces declared constraints; A, I and D are what let those invariants survive concurrency and crashes.*"

Basic syntax you must be able to write cold

```
BEGIN;                                  -- SQL Server: BEGIN TRANSACTION
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;                                 -- or ROLLBACK;

SAVEPOINT s1;  ...  ROLLBACK TO SAVEPOINT s1;     -- partial rollback (SQL Server: SAVE TRANSACTION)
```

- **Autocommit:** outside an explicit `BEGIN`, each statement is its own transaction.
- **Postgres:** an error inside a transaction **aborts the whole transaction** ("current transaction is aborted, commands ignored until end of transaction block") — you must `ROLLBACK` or roll back to a savepoint. **SQL Server** by default continues after a statement-level error unless `SET XACT_ABORT ON` — which you should always enable in stored procedures.
- **Transactions are not free:** they hold locks/snapshots and (in Postgres) prevent vacuum from removing old row versions. Long-running transactions are an operational hazard (§12).

---

2. Durability internals — WAL, checkpoints, crash recovery

Write-ahead logging rule: a data page may be written to disk only *after* the log record describing the change has been written to disk. And COMMIT returns only after its log record is durable.

```
UPDATE → modify page in buffer pool (memory)  → append record to WAL buffer
COMMIT → flush WAL to disk (fsync) → return "committed"     ← durability point
(later) background writer / checkpoint writes dirty data pages to disk
```

Why this is fast: the WAL is **sequential, append-only** I/O; the data files are random I/O and can be written lazily. Many commits can share one fsync (**group commit**).

Crash recovery (ARIES-style; SQL Server literally ARIES; Postgres similar):

1. **Analysis** — find which transactions were active and which pages were dirty at the crash.
2. **Redo** — replay the log forward from the last checkpoint so *committed* changes are on disk (even if their data pages never were).
3. **Undo** — roll back transactions that were in flight and never committed.

Checkpoint = flush dirty pages and record a point recovery can start from → bounds recovery time.

Knobs people ask about

- Postgres `synchronous_commit = off` → COMMIT returns before the WAL fsync: faster, can lose the last few hundred ms of commits on crash (but **never corrupts** — durability weakens, consistency doesn't).
- Replication is built on shipping the WAL/log: **physical** (byte-level, replica identical) vs **logical** (row changes, enables CDC and cross-version replication). Postgres `wal_level=logical` is what Debezium-style change-data-capture reads (Hello Interview has a CDC deep dive).
- **Torn pages / full-page writes**, fsync lies from disks/controllers — the "why durability is harder than it looks" tangent.

⚠ **Durability ≠ replication.** A committed write on the primary is durable on the primary's disk. Whether it survives *losing the primary* depends on **synchronous vs asynchronous replication** (§11): async means a failover can lose the last acknowledged commits.

---

3. Isolation levels and the anomalies — the heart of the round

3.1 The anomalies, defined precisely

Give each as: *interleaving → wrong result.* T1, T2 are concurrent.

Anomaly	Interleaving	Why it's wrong
Dirty write	T1 writes x; T2 overwrites x before T1 commits	Rollback of T1 clobbers T2 (ruled out at *every* standard level)
Dirty read	T1 writes x (uncommitted); T2 reads it; T1 rolls back	T2 acted on data that never existed
Non-repeatable read	T1 reads x; T2 updates x and commits; T1 reads x again → different value	Same query, same txn, different answer
Phantom read	T1 `SELECT … WHERE status='open'` (n rows); T2 inserts a matching row and commits; T1 re-runs → n+1 rows	The *set* of rows changed, not an existing row
Lost update	T1 and T2 both read balance=100; T1 writes 150; T2 writes 130 (computed from 100)	T1's update silently overwritten
Read skew	T1 reads A (old); T2 transfers A→B and commits; T1 reads B (new) → sum is inconsistent	A snapshot taken at *different times* for different rows
Write skew	Both read an invariant ("≥1 doctor on call"); each sees 2; each removes *themselves*; both commit → 0 on call	Neither write conflicts with the other (different rows), yet the **invariant breaks**

⚠ **Write skew is the one that separates people.** It is *allowed* under **snapshot isolation** (what Postgres calls REPEATABLE READ and SQL Server calls SNAPSHOT) and only prevented by **SERIALIZABLE** or by explicitly locking the rows/predicate you read.

3.2 The SQL-standard levels (and why the standard is misleading)

Level	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	❌	possible	possible
REPEATABLE READ	❌	❌	possible *(per standard)*
SERIALIZABLE	❌	❌	❌

The standard was written around lock-based implementations. **Real databases don't match the table:**

Engine	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
PostgreSQL (default RC)	Each *statement* sees a fresh snapshot of committed data. No dirty reads, ever (READ UNCOMMITTED behaves as RC).	**Snapshot isolation**: one snapshot for the whole transaction. **No phantoms** (stricter than the standard). Detects lost update → raises `could not serialize access due to concurrent update`. **Write skew still possible.**	**SSI** (serializable snapshot isolation): snapshot + tracking of read/write dependencies; aborts one transaction on a dangerous cycle with `serialization_failure` (SQLSTATE `40001`). Must retry.

Engine	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
SQL Server (default RC)	Default is **lock-based**: readers take shared locks and *block* writers (and vice-versa). With `READ_COMMITTED_SNAPSHOT ON` (RCSI) it becomes row-versioned like Postgres. **Azure SQL Database enables RCSI by default.**	Lock-based: shared locks held to end of txn; phantoms still possible.	Lock-based: adds **key-range locks** to block phantoms → lots of blocking and deadlocks.
SQL Server **SNAPSHOT**	—	A separate level, row-versioned, whole-transaction snapshot (like Postgres RR); update conflicts raise error 3960. Needs `ALLOW_SNAPSHOT_ISOLATION ON`.	
MySQL InnoDB (default RR)		Snapshot for plain reads, plus **gap/next-key locks** for locking reads	Everything becomes locking reads

Say it this way: "*The standard level names are a contract about which anomalies you can't see, not about how it's implemented. Postgres gives me snapshot-based RC and RR; I only get true serializability at SERIALIZABLE, where I must be prepared to retry on 40001. SQL Server's default is lock-based RC unless RCSI is on — which is the first thing I'd check in an Azure SQL context.*"

⚠ **Postgres `READ UNCOMMITTED` is `READ COMMITTED`.** SQL Server's `READ UNCOMMITTED` / `NOLOCK` is real and *does* allow dirty reads (and can even skip or double-count rows during page splits) — `NOLOCK` everywhere is a classic production smell.

3.3 Which level prevents which anomaly — the table to be able to reproduce

Anomaly	RC (Postgres)	RR / SNAPSHOT	SERIALIZABLE
Dirty read	✅ prevented	✅	✅
Non-repeatable read	❌ possible	✅	✅
Phantom	❌ possible	✅ (Postgres) / ❌ (SQL Server RR)	✅
Lost update	❌ possible *(read-modify-write in app code)*	✅ detected → error	✅
Read skew	❌ possible	✅	✅
Write skew	❌ possible	❌ **possible**	✅

Atomic single-statement update is safe even at RC: `UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;` — the row lock taken by the UPDATE serializes it and the predicate is re-checked after the lock is acquired (Postgres re-evaluates against the newest committed version under RC). **The lost update only occurs when the read and the write are separate statements with application logic between them.** This is the single most useful practical fact in the chapter.

Try it yourself (two psql sessions):

1. Create `accounts(id, balance)`. Under RC: A reads balance, B updates and commits, A re-reads → see the non-repeatable read. Repeat with `BEGIN ISOLATION LEVEL REPEATABLE READ` → value stable.
2. Reproduce lost update with read-then-write in two sessions; then fix it three ways (§5).
3. Reproduce **write skew** with the on-call example under RR; then make it fail under SERIALIZABLE and observe the `40001`.

---

4. MVCC — how snapshots work without readers blocking writers

Idea: never overwrite in place; keep multiple versions of a row; each transaction reads the version visible to *its snapshot*. **Readers don't block writers; writers don't block readers.** Writers still block *other writers of the same row*.

4.1 Postgres MVCC (know this one in detail)

- Every row version (**tuple**) has hidden system columns `xmin` (txid that created it) and `xmax` (txid that deleted/updated it, else 0).
- `UPDATE` = mark the old tuple's `xmax`, **insert a new tuple**. The old one remains until vacuumed.
- **Snapshot** = (xmin, xmax, list of in-progress txids) at statement start (RC) or transaction start (RR/SERIALIZABLE). A tuple is visible if its creator committed before the snapshot and its deleter hadn't (as far as the snapshot can tell). Commit status lives in `pg_xact` (the "clog") and **hint bits** cache it on the tuple.
- **Dead tuples → table bloat. VACUUM** reclaims dead tuples (making space reusable inside the table); **autovacuum** runs it automatically; `VACUUM FULL` rewrites the table (exclusive lock).
- **HOT updates** (heap-only tuples): if the update changes no indexed column and the new version fits on the same page, no new index entries are needed — a large write-amplification win. Hence **indexing a frequently-updated column has a real cost.**
- **Transaction-ID wraparound:** txids are 32-bit; Postgres must periodically **freeze** old tuples. If vacuum can't keep up, the database eventually refuses writes to protect itself. Long-running transactions and abandoned replication slots **hold back vacuum**.
- ⚠ **A long-open transaction (even idle: "idle in transaction") pins the oldest-visible snapshot → nothing newer can be vacuumed → bloat, slow queries.** Monitor `pg_stat_activity.xact_start`.

4.2 SQL Server MVCC (RCSI / SNAPSHOT)

- Row versions are stored in the **version store in `tempdb`** (Azure SQL: persistent version store / Accelerated Database Recovery). Old data isn't kept in the table itself, so no table bloat, but `tempdb` can grow under long snapshot transactions.
- Without RCSI/SNAPSHOT, SQL Server is pure **lock-based**: readers and writers block each other.

4.3 Why MVCC doesn't solve everything

It gives consistent *reads*. It does not stop **write–write conflicts** (resolved by locks/aborts) or **write skew** (needs serializable or explicit locks). Hence the next two chapters.

---

5. Concurrency control — the four tools, and how to choose

This is Hello Interview's "*Dealing with Contention*" pattern in database terms. Four tools:

5.1 Atomic / conditional writes (the first thing to reach for)

```
-- decrement inventory only if available; check rows affected
UPDATE inventory SET qty = qty - 1 WHERE sku = $1 AND qty > 0;     -- 0 rows → sold out
-- guarded state transition
UPDATE orders SET status = 'PAID' WHERE id = $1 AND status = 'PENDING';
```

No read-then-write, no race; the row lock serializes concurrent callers. **Single-row, single-statement invariants should almost always use this.** It is also the safe way to implement an idempotent state machine.

5.2 Pessimistic locking — lock before you read-modify-write

```
BEGIN;
SELECT * FROM inventory WHERE sku = $1 FOR UPDATE;      -- row lock; others wait
-- ...application logic...
UPDATE inventory SET qty = qty - 1 WHERE sku = $1;
COMMIT;
```

- **Lock modes (Postgres):** `FOR UPDATE` (exclusive; blocks other updaters and lockers), `FOR NO KEY UPDATE` (what plain UPDATE takes — doesn't block FK-checking `FOR KEY SHARE`), `FOR SHARE`, `FOR KEY SHARE`. **SQL Server:** `UPDLOCK, ROWLOCK` hints (+ `HOLDLOCK` for range).
- **`FOR UPDATE NOWAIT`** → fail immediately if locked; **`SKIP LOCKED`** → skip locked rows — the standard way to build a **job queue** on a table: `sql SELECT id FROM jobs WHERE state='queued' ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED;` (SQL Server equivalent: `WITH (UPDLOCK, READPAST, ROWLOCK)`.)
- **Use when** contention is high and conflicts are expensive (retrying wastes real work), or you must serialize a multi-step read-modify-write across rows.
- **Costs:** blocking, lock convoys, **deadlocks**, held locks across slow work (⚠ never hold a transaction open across an external call — your own rule).

5.3 Optimistic concurrency control (OCC) — detect conflict at write time

Version column / compare-and-swap:

```
UPDATE doc SET body = $new, version = version + 1
WHERE id = $id AND version = $read_version;     -- 0 rows affected → someone else won → retry/merge
```

Alternatives: compare a `row_version`/`rowversion` (SQL Server) / `xmin` (Postgres, hacky) / ETag/`updated_at`. **Use when** conflicts are rare and reads dominate; no locks are held while users "think". **Cost:** retries under high contention (can livelock/thrash). **ABA problem:** value goes A→B→A so a value-compare thinks nothing changed — use a monotonically increasing version, not the value, as the check.

5.4 Higher isolation as the concurrency tool

`SERIALIZABLE` (Postgres SSI) lets you write simple read-then-write code and have the DB abort dangerous interleavings: **correct by default, but you must wrap the transaction in a retry loop** for `40001` (and `40P01` deadlock). Throughput falls as contention rises.

5.5 Choosing — the decision table you should be able to recite

Situation	Reach for
Single row, simple invariant (counter, stock, state transition)	**Atomic conditional UPDATE**
Multi-row/multi-step invariant, high contention	**Pessimistic** `FOR UPDATE` (lock in a consistent order!)
Low contention, long think-time (edit forms)	**Optimistic** version column
Invariant spans rows you *read but don't write* (write skew)	**SERIALIZABLE**, or lock the rows/parent you read, or a constraint
Invariant expressible as a constraint	**Let the DB enforce it** (UNIQUE, EXCLUDE, CHECK)
Across services/databases	**Distributed lock / lease with fencing**, or redesign to a single owner (§8)
One very hot row	**Shard the counter** (N sub-counters, sum on read) / queue the writes / batch

Constraints beat application checks. "Check then insert" is a race; a `UNIQUE` index is not:

```
INSERT INTO reservations(seat_id, event_id, user_id) VALUES ($1,$2,$3)
ON CONFLICT (event_id, seat_id) DO NOTHING;          -- SQL Server: rely on unique index + catch error 2627/2601
-- Postgres EXCLUDE constraint for "no overlapping bookings":
-- ALTER TABLE booking ADD EXCLUDE USING gist (room_id WITH =, tstzrange(start_at,end_at) WITH &&);
```

5.6 Deadlocks

Definition: T1 holds A and wants B; T2 holds B and wants A. **Detection:** the DB builds a wait-for graph, finds the cycle, **aborts one victim** (Postgres `40P01`; SQL Server error **1205**, chooses the cheaper-to-roll-back victim). **Prevention:** (1) **acquire locks in a consistent global order** (e.g., ascending account id for a transfer); (2) keep transactions short; (3) lock less (row not table; right indexes so a scan doesn't lock more than needed); (4) `NOWAIT`/lock timeouts; (5) **retry** the victim. A non-obvious cause: **missing index** → the UPDATE scans and locks many rows → more cycles. In SQL Server, key-range locks at SERIALIZABLE also deadlock easily.

```
-- transfer without deadlock: always lock the lower id first
SELECT id FROM accounts WHERE id IN ($a,$b) ORDER BY id FOR UPDATE;
```

---

6. Locking in detail

Granularity: row → page → table (and database). **Lock escalation (SQL Server):** after ~5,000 locks on one object, SQL Server may escalate row/page locks to a table lock → sudden blocking. Postgres doesn't escalate; it has no practical limit on row locks (they live on the tuple).

Lock modes & compatibility: Shared (S), Exclusive (X), Update (U — SQL Server's "I intend to update; only one U at a time", prevents the S→X conversion deadlock), Intent locks (IS/IX/SIX — table-level signals that rows below are locked, so a table lock can be granted/denied quickly).

Two-phase locking (2PL): acquire locks as you go (growing), release only at commit/rollback (shrinking). **Strict 2PL** holds all exclusive locks to the end → serializable. This is the lock-based path to SERIALIZABLE.

Phantom protection by locking: predicate locks (lock "all rows matching WHERE…") are expensive; practical engines use **key-range / next-key / gap locks** on index ranges (SQL Server SERIALIZABLE, InnoDB RR). Postgres instead uses SSI's `SIREAD` predicate tracking — no blocking, but may abort.

Advisory locks (Postgres): application-defined locks keyed by an integer (`pg_advisory_xact_lock(key)`), released at transaction end — a cheap *mutex* for "only one worker processes tenant X". (SQL Server: `sp_getapplock`.) Great for serializing a resumable job per `thread_id` — you faced exactly this in ResolveFlow's duplicate-resume bug.

Blocking chains: one stuck transaction holding a lock makes everyone behind it queue; the symptom is "everything is slow, CPU is idle." Diagnose with `pg_locks` + `pg_stat_activity` (`pg_blocking_pids()`), or SQL Server `sys.dm_tran_locks` / `sys.dm_os_waiting_tasks` / `sp_whoisactive`.

---

7. Indexes and query performance — the other half of "SQL in detail"

7.1 B-tree (the default)

Balanced tree, sorted keys, leaf pages linked; lookup/range/ORDER BY in O(log n). Supports `=`, `<`, `>`, `BETWEEN`, `LIKE 'abc%'` (prefix only), `ORDER BY`, and min/max.

- **Composite index = leftmost-prefix rule.** Index `(a, b, c)` serves `a`, `a,b`, `a,b,c` — **not** `b` alone. **Order the columns: equality predicates first, then the range/sort column.** `WHERE tenant_id=? AND status=? AND created_at > ? ORDER BY created_at` → `(tenant_id, status, created_at)`.
- **Selectivity & cardinality:** index helps when it narrows to a small fraction of rows. A boolean or low-cardinality column alone is a poor index; the optimizer will prefer a seq scan when you'd read a large part of the table anyway (random I/O vs sequential).
- **Covering index** (`INCLUDE (cols)` in both Postgres 11+ and SQL Server): leaf holds the extra columns → **index-only scan**, no table lookup. (Postgres still checks the **visibility map**; a recently-updated page forces a heap fetch.)
- **Partial/filtered index:** `CREATE INDEX … WHERE status='queued'` (SQL Server: filtered index) — small, fast, ideal for "pending work" queues.
- **SARGability:** predicates must be *search-argument-able* or the index can't be used. ❌ `WHERE DATE(created_at)=…`, `WHERE lower(email)=…` (without a matching expression index), `WHERE col + 1 = …`, implicit type conversion (e.g., comparing `varchar` to `nvarchar` in SQL Server — a classic scan trigger), leading-wildcard `LIKE '%x'`.
- **Write cost:** every index is maintained on every insert/delete and on updates of indexed columns (and defeats HOT). Indexes aren't free; "add an index" isn't automatically the answer.
- **Clustered vs heap: SQL Server** tables are usually a **clustered index** (the table *is* the B-tree on the clustered key; non-clustered indexes store the clustered key as the row locator — so a wide clustered key bloats every index; prefer narrow, ever-increasing keys). **Postgres** tables are **heaps**; every index points to a physical tuple id (ctid); `CLUSTER` is a one-time reorder, not maintained.
- **Random UUID primary keys** fragment B-trees (inserts land all over the tree → page splits, poor cache locality). Use sequential/time-ordered ids (UUIDv7/ULID, `NEWSEQUENTIALID()`, identity).

7.2 Other index types (name them, know when)

Hash (equality only), **GIN** (inverted — arrays, JSONB, full-text), **GiST/SP-GiST** (geometric, ranges, nearest-neighbor, EXCLUDE), **BRIN** (tiny block-range summaries for huge, naturally-ordered tables like time-series), **columnstore** (SQL Server — analytics), **full-text**, **vector** (pgvector HNSW/IVFFlat — your RAG work).

7.3 Reading a query plan

`EXPLAIN (ANALYZE, BUFFERS)` (Postgres) / **Actual Execution Plan** (SQL Server). Know:

- **Access paths:** Seq Scan / Index Scan / Index-Only Scan / Bitmap Heap Scan (SQL Server: Table Scan, Index Seek vs Index **Scan**, Key Lookup / RID Lookup).
- **Join algorithms:** | Algorithm | How | Best when | |---|---|---| | **Nested loop** | for each outer row, probe inner (ideally via index) | small outer, indexed inner | | **Hash join** | build hash table on smaller input, probe with larger | large, unsorted, equality joins | | **Merge join** | both inputs sorted on the join key, merge | both already ordered (or index-ordered) |
- **Estimated vs actual rows:** a big mismatch = stale/missing **statistics** (`ANALYZE`; `UPDATE STATISTICS`) or correlated predicates → bad plan. **This is the #1 diagnostic.**
- **Cost-based optimizer:** chooses among plans using statistics (histograms, distinct counts). SQL Server **parameter sniffing**: the plan is compiled for the first parameter values and cached; later values with a different distribution run the wrong plan (mitigations: `OPTION (RECOMPILE)`, `OPTIMIZE FOR`, query-store plan forcing). Postgres generic vs custom plans for prepared statements.
- **N+1 queries:** one query for N parents then N queries for children → fix with a join / `IN (…)` batch / `ANY($1::int[])`. **OFFSET pagination** is O(offset) — use **keyset (seek) pagination**: `WHERE (created_at, id) < ($last_ts, $last_id) ORDER BY created_at DESC, id DESC LIMIT 50`.
- **Triage order for "this query is slow":** (1) get the actual plan; (2) compare estimated vs actual rows; (3) is there a seq scan/index scan on a big table that should be a seek? (4) are statistics fresh? (5) is the predicate SARGable and types matching? (6) missing/ill-ordered composite index? (7) is it blocked (waiting on a lock) rather than slow? (8) only then rewrite/denormalize/cache.

8. Transactions across systems — 2PC, sagas, outbox, idempotency

A single-database transaction can't span services. Know the options and the honest trade-offs:

8.1 Two-phase commit (2PC / XA / MSDTC)

Phase 1 (prepare): coordinator asks every participant "can you commit?" — each durably logs *prepared* and votes yes/no. **Phase 2 (commit/abort):** coordinator decides and tells all.

- **Atomic across participants**, **but blocking**: if the coordinator dies after prepare, participants holding locks are **in doubt** and must wait. Coordinator is a SPOF; latency = 2 round trips + fsyncs; poor availability under partitions (CAP: it chooses C over A). Used inside databases / between homogeneous resource managers (SQL Server **MSDTC**, Postgres `PREPARE TRANSACTION`), rarely across microservices and not at all across third-party APIs.

8.2 Sagas — a sequence of local transactions with compensations

```
reserve inventory ─▶ charge card ─▶ create shipment ─▶ done
       ▲ compensate: release   ▲ compensate: refund   ▲ compensate: cancel
```

- **Orchestration** (a coordinator drives steps and compensations — the durable-execution style: Temporal, Durable Functions, Step Functions, your LangGraph checkpointer) vs **choreography** (services react to each other's events; harder to see/own the flow).
- **Not isolated:** other transactions see intermediate states (ACD without I) → handle with **semantic locks** (`status=PENDING`), commutative updates, reread-and-verify.
- **Compensations must be idempotent and can fail** → retry forever / alert a human.
- **Pivot step:** the point after which you can only go forward.

8.3 The dual-write problem and the transactional outbox

Writing to the DB **and** publishing to a broker are two systems; a crash between them loses or duplicates one. **Outbox:** in the *same local transaction* that changes state, insert an `outbox(event_id, payload, …)` row. A separate relay (poller or **CDC** on the WAL → Kafka) publishes it **at-least-once**; consumers dedupe. This converts "atomic across two systems" into "atomic locally + reliable at-least-once delivery."

```
BEGIN;
  UPDATE orders SET status='PAID' WHERE id=$1 AND status='PENDING';
  INSERT INTO outbox(event_id, topic, payload) VALUES (gen_random_uuid(), 'order.paid', $2);
COMMIT;        -- relay publishes later; consumers dedupe on event_id
```

8.4 Idempotency — the answer to "at-least-once"

Exactly-once delivery doesn't exist across a network; **exactly-once *effect* = at-least-once delivery + idempotent processing.** Mechanisms: **idempotency key** with a UNIQUE constraint (`INSERT … ON CONFLICT DO NOTHING RETURNING` — only the inserter proceeds); natural-key upserts; conditional state transitions (§5.1); versioned writes; **inbox table** of processed message ids (insert + business change in one transaction). Payment APIs (Stripe-style) store the key *with the response* so a retry returns the original result.

8.5 Distributed locks and fencing

Redis `SET key val NX PX ttl` or ZooKeeper/etcd leases. **A lock with a TTL is a *lease*; a paused process can resume after expiry believing it still holds the lock.** Defend with a **fencing token** (monotonic number issued with the lease; the *resource* rejects writes with a stale token). Redlock's safety is contested; for correctness prefer a database row/constraint or a consensus store.

---

9. SQL fluency — the language, in detail

9.1 Logical order of evaluation (explains most "why doesn't this work?")

`FROM/JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT (aliases) → DISTINCT → ORDER BY → LIMIT/OFFSET/TOP`. So: you can't use a SELECT alias in WHERE (Postgres allows it in ORDER BY/GROUP BY), and window functions can't be used in WHERE (wrap in a CTE/subquery).

9.2 Joins

INNER, LEFT/RIGHT/FULL OUTER, CROSS, **SEMI** (`EXISTS`), **ANTI** (`NOT EXISTS`), self-join. ⚠ **`WHERE` on the right table of a LEFT JOIN silently turns it into an INNER JOIN** — put the condition in `ON`. ⚠ **`NOT IN (subquery)` returns nothing if the subquery yields any NULL** — use `NOT EXISTS`.

9.3 NULL and three-valued logic

`NULL = NULL` is UNKNOWN, not true; `WHERE x <> 5` excludes NULL rows; aggregates ignore NULL (`COUNT(*)` counts rows, `COUNT(col)` counts non-NULL); `NULL` in `UNIQUE`: Postgres allows multiple NULLs (use `NULLS NOT DISTINCT` in 15+), SQL Server allows exactly one. Use `IS [NOT] DISTINCT FROM`, `COALESCE`.

9.3a GROUP BY / HAVING

`WHERE` filters rows *before* grouping; `HAVING` filters groups *after*. Every non-aggregated SELECT column must be in GROUP BY.

9.4 Window functions (heavily tested)

```
-- top-N per group
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn FROM emp) t WHERE rn <= 3;
-- running total
SUM(amount) OVER (PARTITION BY account ORDER BY ts ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
-- previous row / delta
amount - LAG(amount) OVER (PARTITION BY account ORDER BY ts)
```

`ROW_NUMBER` (unique 1,2,3), `RANK` (ties share, gaps), `DENSE_RANK` (ties share, no gaps), `LAG/LEAD`, `NTILE`, `FIRST_VALUE`. The **frame** (`ROWS` vs `RANGE`) matters; the default frame with `ORDER BY` is `RANGE UNBOUNDED PRECEDING … CURRENT ROW` (peers included) — a common source of wrong running totals.

9.5 CTEs, recursion, set ops, upsert

```
WITH RECURSIVE tree AS (
  SELECT id, parent_id, 1 AS depth FROM org WHERE parent_id IS NULL
  UNION ALL
```

```
  SELECT o.id, o.parent_id, t.depth+1 FROM org o JOIN tree t ON o.parent_id = t.id)
SELECT * FROM tree;                          -- SQL Server: omit RECURSIVE
-- upsert
INSERT INTO t(k,v) VALUES ($1,$2) ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v;   -- SQL Server: MERGE
(careful: race-prone, hint HOLDLOCK) or UPDATE-then-INSERT in a txn
-- claim-and-return
UPDATE jobs SET state='running' WHERE id = (SELECT id FROM jobs WHERE state='queued' LIMIT 1 FOR UPDATE
SKIP LOCKED) RETURNING *;   -- SQL Server: OUTPUT
```

`UNION` dedupes (sort/hash cost); `UNION ALL` doesn't — default to `ALL`. `INTERSECT`, `EXCEPT`. Postgres CTEs are inlined since v12 unless `MATERIALIZED`.

9.6 Subqueries: correlated vs uncorrelated; `EXISTS` vs `IN` (equivalent for non-NULL; `EXISTS` short-circuits).

9.7 Dialect map (Postgres ↔ SQL Server) — the thing that trips candidates

Concept	PostgreSQL	SQL Server (T-SQL)
Limit rows	`LIMIT n OFFSET m`	`TOP n` / `OFFSET m ROWS FETCH NEXT n ROWS ONLY`
String concat	`||`	`+` / `CONCAT`
Auto id	`GENERATED … AS IDENTITY` / `serial`	`IDENTITY(1,1)` / `SEQUENCE`
Upsert	`ON CONFLICT`	`MERGE`
Return changed rows	`RETURNING`	`OUTPUT inserted.*`
Boolean	`boolean`	`bit`
Date now	`now()`	`SYSUTCDATETIME()`
Temp table	`CREATE TEMP TABLE`	`#temp` / `@tablevar`
Procedural	PL/pgSQL	T-SQL (`TRY…CATCH`, `THROW`, `SET XACT_ABORT ON`)
JSON	`jsonb` + GIN	`JSON_VALUE`, `OPENJSON`, `nvarchar(max)` / native `json`
Isolation default	READ COMMITTED (MVCC)	READ COMMITTED (lock-based) *unless RCSI*
Case sensitivity	case-sensitive text	collation-dependent (default CI)

9.8 Practice problems — write each yourself, then compare plans

1. Second-highest salary per department (handle ties and departments with one employee).
2. Users whose purchases rose three months in a row (window + LAG).
3. Sessionization: group events into sessions with a 30-minute gap (LAG + running SUM of flags).
4. Find duplicates and delete all but the newest (ROW_NUMBER; CTE delete).
5. Gaps-and-islands: consecutive-day login streaks.
6. Median per group without a MEDIAN function (`PERCENTILE_CONT`).
7. Hierarchy: all reports (transitive) of a manager (recursive CTE) — and detect cycles.
8. Pivot rows → columns with conditional aggregation.

---

10. Schema design, normalization, and keys

- **Normalization:** 1NF (atomic values, no repeating groups), 2NF (no partial dependency on part of a composite key), 3NF (no transitive dependency — "every non-key attribute depends on the key, the whole key, and nothing but the key"), BCNF. **Denormalize deliberately** for read paths (precomputed counts,

materialized views, duplicated columns) and own the consistency cost (update both, or derive via triggers/CDC/async).
- **Keys:** natural vs surrogate; composite PKs; **foreign keys** enforce integrity (cost: lock/check on writes, **index the FK column** — missing FK index is a top cause of slow deletes and deadlocks; `ON DELETE CASCADE/RESTRICT`).
- **Constraints as design:** `CHECK`, `UNIQUE`, partial unique indexes (`UNIQUE … WHERE deleted_at IS NULL`), `EXCLUDE`. Soft-delete interacts badly with uniqueness — know the partial-index fix.
- **Multi-tenancy (Karan-adjacent):** shared schema with `tenant_id` on every table + composite indexes leading with `tenant_id` + **Row-Level Security** (Postgres `CREATE POLICY`; SQL Server RLS security policies) vs schema-per-tenant vs database-per-tenant (isolation vs ops cost).
- **Time:** store UTC (`timestamptz`/`datetimeoffset`), never local; money as integer minor units or `numeric`/`decimal`, never float.
- **Data types & migrations:** adding a NOT NULL column with a default (rewrites table pre-PG11 / metadata-only now), `CREATE INDEX CONCURRENTLY` (Postgres) / `ONLINE = ON` (SQL Server Enterprise) to avoid blocking writes; **expand → migrate → contract** for zero-downtime schema change.

---

11. Scaling the data layer — replication, partitioning, sharding, consistency

11.1 Replication

- **Single-leader (primary/replicas):** writes to leader; replicas stream the log. **Async** = fast but **replication lag** (stale reads, lost writes on failover); **sync** = durable, slower, availability hit if the sync replica is down; **semi-sync / quorum** (Postgres `synchronous_standby_names ANY 1`, SQL Server **Always On Availability Groups** sync vs async commit mode).
- **Read-your-writes / monotonic reads** anomalies from lag → mitigations: route a user's reads to the leader for N seconds after a write, session tokens / LSN-based reads, or read from leader for critical paths.
- **Multi-leader / leaderless (Dynamo, Cassandra):** conflict resolution (LWW, vector clocks, CRDTs), quorum `W + R > N` — tie to CAP (your own `CAP_THEOREM_AND_AGENTIC_SYSTEMS.md`).
- **Failover:** detect → promote → repoint → fence the old leader (STONITH) to avoid **split brain**.

11.2 Partitioning vs sharding

- **Partitioning** = split one table inside one database (range by date, list, hash) → partition pruning, cheap retention (`DROP PARTITION`), smaller indexes. **Sharding** = split across databases/nodes.
- **Shard key choice is the whole game:** high cardinality, even distribution, matches the dominant access pattern (so most queries hit one shard); avoid **hot shards** (monotonic keys, celebrity tenants); cross-shard joins/transactions become your problem (→ sagas, denormalization).
- **Range vs hash vs directory sharding; consistent hashing** (minimal data movement when nodes are added — Hello Interview has a page); **resharding** is the painful operation → plan virtual shards / many logical shards over few physical nodes.
- Tenancy-aware sharding: pin large tenants to dedicated shards.

11.3 Caching and the consistency it costs

Cache-aside / read-through / write-through / write-behind; **invalidation is the hard part**; stampede / thundering herd (request coalescing, jittered TTL, locking); stale-read tolerance must be an explicit product decision. Cache is **never** the source of truth for an invariant.

11.4 Numbers worth knowing (order of magnitude — Hello Interview "Numbers to Know")

Single Postgres/SQL node: thousands to low tens of thousands of simple TPS depending on hardware and fsync; a well-indexed point read ~ sub-millisecond-to-ms from cache, ~ms from SSD; sequential vs random I/O differ by

orders of magnitude; fsync latency (≈ 0.1–2 ms SSD, more on network disks) bounds single-connection commit rate → **group commit/batching** is the lever. Connection count is a limit (Postgres process-per-connection → use a **pooler**, PgBouncer; SQL Server uses workers/threads).

---

12. Operating the data layer

- **Connection pooling:** pooled connections + transaction-mode pooling breaks session state (advisory locks, `SET`, prepared statements) — know the caveat.
- **Slow query / lock diagnostics:** `pg_stat_statements`, `auto_explain`, `pg_stat_activity`, `pg_locks`; SQL Server **Query Store**, DMVs, Extended Events, `sp_whoisactive`, deadlock graph.
- **Long transactions:** the three-way hazard — hold locks, pin MVCC snapshots (bloat / tempdb), block DDL (a `ALTER TABLE` waiting for a lock queues *all* later queries behind it → set `lock_timeout`).
- **Backups & recovery:** full + incremental + **PITR (point-in-time recovery)** from base backup + archived WAL (Azure SQL: automated PITR); **RPO** (data loss tolerated) vs **RTO** (downtime tolerated); a backup you've never restored isn't a backup.
- **Retry policy:** retry only **idempotent** operations or whole transactions on `40001/40P01/1205`; exponential backoff + jitter; cap attempts.
- **Observability:** per-query latency percentiles, lock wait time, replication lag, bloat/dead tuples, connection saturation, checkpoint frequency, cache hit ratio.

---

13. Worked design problems — transactions in the wild

Narrate each with the eight beats from §16.1 of `GITLAB_KARAN_BHATIA_DEEP_DIVE`: invariant → race → tool → failure mode → test.

13.1 Bank transfer

Invariant: `sum(balances)` conserved, no negative balance. One DB: single transaction; lock in id order (`FOR UPDATE ... ORDER BY id`); `CHECK (balance >= 0)`; **ledger table** (double-entry, append-only) as the source of truth with balances derived or checked — immutable audit trail. Idempotency key per transfer. Across banks: saga with a "pending" state and reconciliation.

13.2 Flash sale / inventory without overselling

Race: N buyers, `qty=1`. Naive read-check-decrement oversells. Fixes in order: **atomic `UPDATE … SET qty=qty−1 WHERE qty>0`** (check rows affected) → hot-row contention → **reserve then confirm** with TTL (reservation rows, expiry sweeper) → very hot SKU: **pre-shard the stock into k buckets** (a buyer picks a bucket, falls back), or **queue/admission control** in front, or Redis atomic `DECR` with async reconcile to the DB. State the oversell vs availability trade explicitly.

13.3 Ticket / seat booking

`UNIQUE(event_id, seat_id)` + `INSERT … ON CONFLICT DO NOTHING`; a **hold** (status `HELD`, `expires_at`) before payment; sweeper releases expired holds; payment confirmation flips `HELD→SOLD` with a conditional update guarded by hold owner.

13.4 Payments

Idempotency keys; ledger; **state machine** with conditional transitions; outbox for events; reconciliation against the provider; never trust the 200 — read back / webhook-confirm. (Hello Interview "Payment System".)

13.5 Job queue on a table

`FOR UPDATE SKIP LOCKED` claim + **lease** (`locked_until`) + heartbeat; **visibility timeout**; retries with a `attempts` counter and dead-letter state; unique `dedupe_key`. (Your ResolveFlow thread ownership + approval-resume fix is a first-class example: *claim a key under a unique constraint, only the inserter proceeds.*)

13.6 Rate limiter / counters

Single Redis node `INCR` + `EXPIRE` (or Lua) / token bucket; in SQL it's a hot row → shard the counter. (Your `RATE_LIMITING_AGENTIC_SYSTEMS.md`.)

13.7 Collaborative edit / "last write wins" vs OCC

Version column + conflict UI, or CRDT/OT for real-time merge.

---

14. The question bank — answer each aloud, then check

Fundamentals

1. Explain ACID with an example for each letter. Which are the database's job and which is yours?
2. What does the WAL do, and why can the database acknowledge COMMIT before data pages hit disk?
3. Walk through crash recovery (analysis / redo / undo).
4. Define dirty read, non-repeatable read, phantom, lost update, write skew with an interleaving each.
5. What does READ COMMITTED guarantee in Postgres vs SQL Server? What is RCSI and why does Azure SQL default to it?
6. Postgres REPEATABLE READ vs the SQL standard — what's different? Does it allow write skew?
7. How does MVCC work? What are `xmin`/`xmax`? What is vacuum and why does bloat happen?
8. What's the difference between snapshot isolation and serializable? Give the example.
9. How does Postgres implement SERIALIZABLE? What must the application do?
10. Why can a long-running transaction hurt a Postgres database that's otherwise idle?

Concurrency & locking

11. Two requests decrement the last item concurrently. Give three correct implementations and their trade-offs.
12. Optimistic vs pessimistic — when each? What's the ABA problem?
13. What is a deadlock; how does the DB detect/resolve it; how do you prevent it in a transfer?
14. `SELECT … FOR UPDATE SKIP LOCKED` — what's it for?
15. What's lock escalation? Intent locks? Update (U) locks?
16. How do you stop phantoms under a lock-based SERIALIZABLE? Key-range locks.
17. "Check then insert" race — how do you fix it properly? (unique constraint, `ON CONFLICT`)
18. Why is holding a transaction open across an HTTP call dangerous?
19. A page is slow and CPU is idle — how do you tell it's lock blocking?
20. Design a mutex for "one worker per tenant." (advisory lock / lease + fencing)

SQL

21. Write: top 3 salaries per department; running total; gaps-and-islands streaks; dedupe keep-newest.
22. `RANK` vs `DENSE_RANK` vs `ROW_NUMBER`.
23. Why does `NOT IN` misbehave with NULLs? What does a WHERE on the right table do to a LEFT JOIN?
24. `WHERE` vs `HAVING`. Logical order of evaluation.
25. `UNION` vs `UNION ALL`. `EXISTS` vs `IN`.
26. Implement upsert in Postgres and SQL Server and state each one's race hazards.

Indexes & performance

27. How does a B-tree index work; what is the leftmost-prefix rule; how do you order a composite index?
28. What is a covering index? Partial/filtered index? Clustered vs non-clustered (heap vs clustered)?
29. A query ignores your index — list the reasons (SARGability, type mismatch, low selectivity, stale stats, function on column).
30. Read this plan: estimated 10 rows, actual 2M — what happened?
31. Nested loop vs hash join vs merge join: when each?
32. Why is OFFSET pagination slow and what replaces it?
33. What's parameter sniffing?
34. Why are random UUID PKs bad for B-trees?

Distributed

35. Why can't you just use a transaction across two services? Explain 2PC and its failure mode (in-doubt blocking).
36. Saga: orchestration vs choreography; how do you handle a failed compensation?
37. The dual-write problem and the outbox pattern — walk through the failure modes it removes.
38. Exactly-once: possible? What do you actually build? (at-least-once + idempotent)
39. A Redis lock with a TTL — what can go wrong; what is a fencing token?
40. Sync vs async replication: durability, availability, lag; how do you give read-your-writes?
41. How do you choose a shard key; what are hot shards; how do you reshard?
42. CAP vs ACID's C — different meanings of "consistency"?
43. How do you do a zero-downtime schema migration?
44. RPO vs RTO; PITR; how do you know your backup works?

Your own work (they will ask)

45. "Tell me about a concurrency bug you actually hit." → ResolveFlow: a paused approval that vanished / duplicate resume → per-thread serialization + idempotent claim (§2.4 of the Karan doc). Know what isolation level and what lock/constraint you used and *why that was sufficient*.
46. "Why Postgres for your checkpointer?" → durable, transactional state; resume from snapshot not event replay; thread ownership row.
47. "How did you make agent actions safe to retry?" → idempotency keys, conditional transitions, outbox.

---

15. What to study on Hello Interview — in order, mapped to this doc

Your dashboard shows **System design 12%, Coding 0%, Behavioral 0%**. For a *transactions / SQL* round the high-yield pages are below. URLs are all under `https://www.hellointerview.com/learn/system-design/`.

Order	Hello Interview page	Path	Maps to chapters here
1	**PostgreSQL** deep dive (incl. its *Appendix: ACID & Basic SQL Concepts*)	`deep-dives/postgres`	1–4, 7, 9
2	**Data Modeling**	`core-concepts/data-modeling`	10
3	**Database Indexing**	`core-concepts/db-indexing`	7
4	**Pattern: Dealing with Contention** — conditional writes, pessimistic locking, OCC, isolation levels, distributed locks, deadlocks, ABA	`patterns/dealing-with-contention`	3, 5, 6, 8.5
5	**Pattern: Multi-step Processes** (sagas / workflows / durable execution)	`patterns/multi-step-processes`	8

Order	Hello Interview page	Path	Maps to chapters here
6	**Key Technologies: Temporal** (durable execution — new, Sep 20)	`deep-dives/temporal`	8.2 (your LangGraph checkpointing maps directly)
7	**Change Data Capture**	`deep-dives/change-data-capture`	2, 8.3
8	**Sharding**, **Consistent Hashing**	`core-concepts/sharding`, `…/consistent-hashing`	11.2
9	**CAP Theorem**	`core-concepts/cap-theorem`	11.1, 14 Q42
10	**Pattern: Scaling Reads / Scaling Writes**	`patterns/scaling-reads`, `…/scaling-writes`	11
11	**Caching**	`core-concepts/caching`	11.3
12	**Numbers to Know**	`core-concepts/numbers-to-know`	11.4
13	**DynamoDB** and **Cassandra** deep dives (NoSQL contrast; Microsoft analogue is Cosmos DB)	`deep-dives/dynamodb`, `…/cassandra`	11.1
14	**Kafka** (outbox/CDC consumer side, delivery semantics)	`deep-dives/kafka`	8.3–8.4
15	**Redis** (locks, counters, rate limiting)	`deep-dives/redis`	8.5, 13.6
16	**ZooKeeper** (leader election / fencing)	`deep-dives/zookeeper`	8.5

Problem breakdowns that exercise transactions (do these, narrating the invariant + race + fix): Ticketmaster, **Payment System**, **Online Auction**, **Robinhood**, **Gopuff** (inventory), **Job Scheduler**, **Bitly** (unique id generation), **Distributed Rate Limiter**; plus the new **Flash Sale** answer key and the **In the Wild: Shopify Inventory Reservations** article (real reservation design) and **Meta ZGateway** (connection pooling / admission control).

Their built-in practice on your dashboard: the *System Design Interview Readiness* checklist (12% done) — finish "Study core system design concepts", "Learn key technologies", and "Learn System Design Patterns" in the order above, then do **3 easy + 3 medium + 2 hard guided practices**, choosing Ticketmaster / Payment / Flash-sale-style problems first. Take the **quizzes** (you've done 2 of 67) on the pages above after reading each one — they test exactly the definitions in Chapters 3, 5 and 7.

> I could read page *headings* on Hello Interview from your logged-in session, not their full text; treat the mapping above as a reading order, and use this document as the mechanism-level source.

---

16. A 7-day plan for this round (adapt to your actual date)

Day	Do	Output
1	Ch. 1–3 here + HI *PostgreSQL* page. **Two-session experiments** for every anomaly in §3.	You can reproduce lost update, write skew, `40001`
2	Ch. 4–6 + HI *Dealing with Contention*. Implement conditional update, `FOR UPDATE`, OCC, `SKIP LOCKED` queue. Cause and read a deadlock.	One working job queue; deadlock graph read

Day	Do	Output
3	Ch. 7 + HI *Indexing*. Make a 1M-row table; `EXPLAIN ANALYZE` 8 queries before/after indexes; break SARGability on purpose.	Plan-reading fluency
4	Ch. 9 + all 8 SQL practice problems (§9.8), in Postgres, then translate two to T-SQL.	Window functions automatic
5	Ch. 8 + HI *Multi-step Processes* / *Temporal* / *CDC*. Write the outbox + idempotent consumer.	Saga + outbox on a whiteboard
6	Ch. 10–12 + HI *Sharding/CAP/Scaling*. Design Flash Sale and Payment (HI answer keys) aloud, timed 35 min.	Two narrated designs
7	Question bank (Ch. 14) aloud, no notes; Microsoft-flavor pass: Azure SQL specifics (RCSI default, Always On, PITR, MSDTC, Cosmos DB basics).	Weak-spot list

Interview posture for this round: for every concurrency answer, name **the invariant**, **the interleaving that breaks it**, **the cheapest tool that fixes it**, and **what the isolation level does *not* protect** — in that order. When you aren't sure what an engine does by default, say what you'd *check* (`SHOW transaction_isolation`, `sys.databases.is_read_committed_snapshot_on`) rather than guess — that reads as senior.
